Decentralised Lending Service - Project Report

1. Architectural Choices

The Decentralised Lending Service has been implemented using a modular and upgradeable architecture on Ethereum.

Upgradeable Pattern:
The production topology uses `FundingPool` as a UUPS implementation and `FundingPoolProxy` as an ERC-1967 proxy. The proxy is initialized atomically with the oracle address, while the implementation disables direct initialization. Therefore upgrades preserve pool state and can be authorized only by the proxy owner; a regression test upgrades to `FundingPoolV2` and verifies that contributor accounting is retained.

Loan Management:
Upon approval, the pool deploys a dedicated `LoanContract`. Vote weights and liquidity are evaluated over the contributors' current disposable balances at resolution; non-voters are rejects. Interest is split into direct gains and compensation collateral. An overdue loan is marked defaulted by a default request or by its first late repayment; compensation reduces the compensated contributor's position, and a failure remains irreversible even if a late repayment follows.

Bitcoin Liquidity Oracle:
The on-chain `BitcoinOracle.sol` is a registry. The Python service (`oracle_service.py`) processes the first 131,000 Bitcoin mainnet blocks one at a time, maintains UTXOs, listens for `UpdateRequested`, and writes the requested balance in satoshi. Its fixed minimum fee is the measured 48,799-gas update cost multiplied by 0.1 gwei.

2. Gas Evaluation Results

During the testing phase, we profiled the main operations of the protocol to understand the gas footprint. The tests were run on the local Hardhat network.

Operation	Gas Used	Notes
`contribute()`	140,945	Registers contributor accounting.
`withdraw()`	47,693	Updates balances and refunds ETH.
`createLoanProposal()`	189,700	Stores a proposal.
`voteOnProposal()`	106,758	Records an approval/rejection vote.
`requestUpdate()`	24,575	Emits the oracle request.
`update()`	48,799	Stores a fresh Bitcoin balance.
`executeApprovedLoan()`	912,836	Weights votes, locks funds, deploys a loan.
`repay()`	183,497	Distributes base, gains and collateral.
`claimInterestGain()`	33,454	Pays a contributor's accrued gain.

Note: `executeApprovedLoan` is the most expensive normal operation because it evaluates current vote weights, locks contributors and deploys a contract. Default, compensation and termination paths are covered by regression tests.

3. Reentrancy Vulnerability & Attack

To demonstrate security risks, we included a deliberately vulnerable loan contract and a dedicated attacker contract.

The Vulnerability:

In the vulnerable version, the `withdraw()` function sends ETH to the user *before* updating their internal balance (`balances[msg.sender] = 0;`). This violates the Checks-Effects-Interactions pattern.

The Attack:

The `ReentrancyAttacker` contract deposits a small amount of ETH and immediately calls `withdraw()`. When the vulnerable contract sends the ETH, it triggers the attacker's `receive()` fallback function. Because the attacker's balance hasn't been set to 0 yet in the vulnerable contract's state, the fallback function can recursively call `withdraw()` again. This loop drains the contract's entire ETH balance before the state is ever updated.

Our production contracts (`FundingPool` and `LoanContract`) protect against this by strictly adhering to the **Checks-Effects-Interactions** pattern, updating state variables *before* any external ETH transfers.

4. Malicious Strategies Discussion

Exploiting the Repayment Sorting Rule

The system's requirements dictate that during a partial repayment, the base amount is split among the participants by refunding the contributors in order of their locked value, **from highest to lowest**.

This introduces a significant economic vulnerability that a malicious actor can exploit without relying on smart contract bugs (like reentrancy).

The Strategy:

1. A malicious actor deposits a massive amount of ETH into the funding pool, ensuring they are the contributor with the absolute highest disposable value.
2. The same actor applies for a very large loan. Because they have a huge disposable value, a massive chunk of their own loan will be funded by their own locked contribution, but a portion will still be funded by other honest contributors.
3. Once the loan is granted, the malicious borrower performs a **partial repayment**. Because refunds are processed from the highest locked value first, the malicious actor's partial repayment goes entirely to refunding their *own* locked contribution in the pool.
4. The malicious actor then intentionally defaults on the remainder of the loan.

Result:

The malicious actor gets all of their original contribution back (via the partial repayment), while simultaneously walking away with the remaining borrowed funds that were sourced from the honest contributors. The honest contributors take the entirety of the loss, as their locked funds were at the bottom of the refund queue.

5. User Manual

Prerequisites

- **Node.js** & **npm**

- **Python 3.10+** (dependencies in `python/requirements.txt`)
- A local **Geth** node running the provided genesis block (Chain ID 202526).

1. Start the Blockchain

Run `./setup.sh`, then initialize and start Geth with the prefunded account unlocked as the Clique miner:

```
./setup.sh
cd private-chain
./geth --datadir=./data init project2526genesis.json
./geth --datadir=./data --keystore=./keystore \
  --networkid=202526 --http --http.addr=127.0.0.1 \
  --http.port=8545 --http.api=eth,net,web3 --mine \
  --miner.etherbase=d278d247a52c550508ea2b2c9321d816238fb523 \
  --unlock d278d247a52c550508ea2b2c9321d816238fb523 \
  --password password.txt --allow-insecure-unlock --ipcdisable
```

2. Run Setup

Execute the Python setup script to generate accounts, deploy the contracts, and create `.env` files:

```
./.venv/bin/python python/setup.py
```

This deploys BitcoinOracle, the FundingPool implementation and its initialized proxy, and generates new keys for the Deployer, Contributor, and Borrower.

3. Start Background Services

Open two new terminals and start the off-chain components. Keep them running:

```
# Terminal 1: Bitcoin Oracle
./.venv/bin/python python/oracle/oracle_service.py

# Terminal 2: Auto Voter
./.venv/bin/python python/voter/auto_voter.py
```

4. Run the Simulation

In a separate terminal, run the full lifecycle simulation script:

```
./.venv/bin/python python/simulate.py
```

This script executes a deposit, proposal, oracle update, vote, resolution, repayment and interest claim. It prints balances, disposable funds, compensation funds, collateral percentage and active-loan count after every meaningful transition.